# Actividad 1

## Objetivo

Comprender los conceptos de la capa física y enlace de datos, según modelo OSI, mediante la utilización de un simulador de circuitos electrónicos que permita construir un sistema de comunicación básico.

## Instrucciones generales

1. La Actividad puede ser realizada en parejas.
2. Lea y comprenda cuidadosamente lo que se le solicite.
3. La Actividad está conformada por **Partes**. En cada **Parte**, podrá comprender ciertos conceptos teóricos dados en la lección.
   a) **Primera parte:** En esta parte, se creará la versión del “Hola Mundo” en el Arduino.
   b) **Segunda parte:** Asimismo, en esta parte se modificará el circuito anterior para que permita realizar una comunicación por medio del Código Morse.
   c) **Tercera parte:** Finalmente, en esta parte se volverá a modificar el circuito previo para hacer que 2 Arduinos se comuniquen, utilizando el Código Morse.

## Requisitos

1. Utilización del simulador Tinkercad. Disponible en el enlace https://www.tinkercad.com/dashboard.
2. Para el requisito anterior se debe crear una cuenta y luego iniciar sesión.

**Inicio de la Actividad.**

## Primera Parte – “Hola Mundo”

1. Ingrese al simulador Tinkercad mediante el siguiente enlace https://www.tinkercad.com/dashboard.
2. Una vez que ha iniciado, se ve similar a como se observa en la siguiente figura. En donde deberá seleccionar la opción de Circuitos (en el menú de la izquierda).

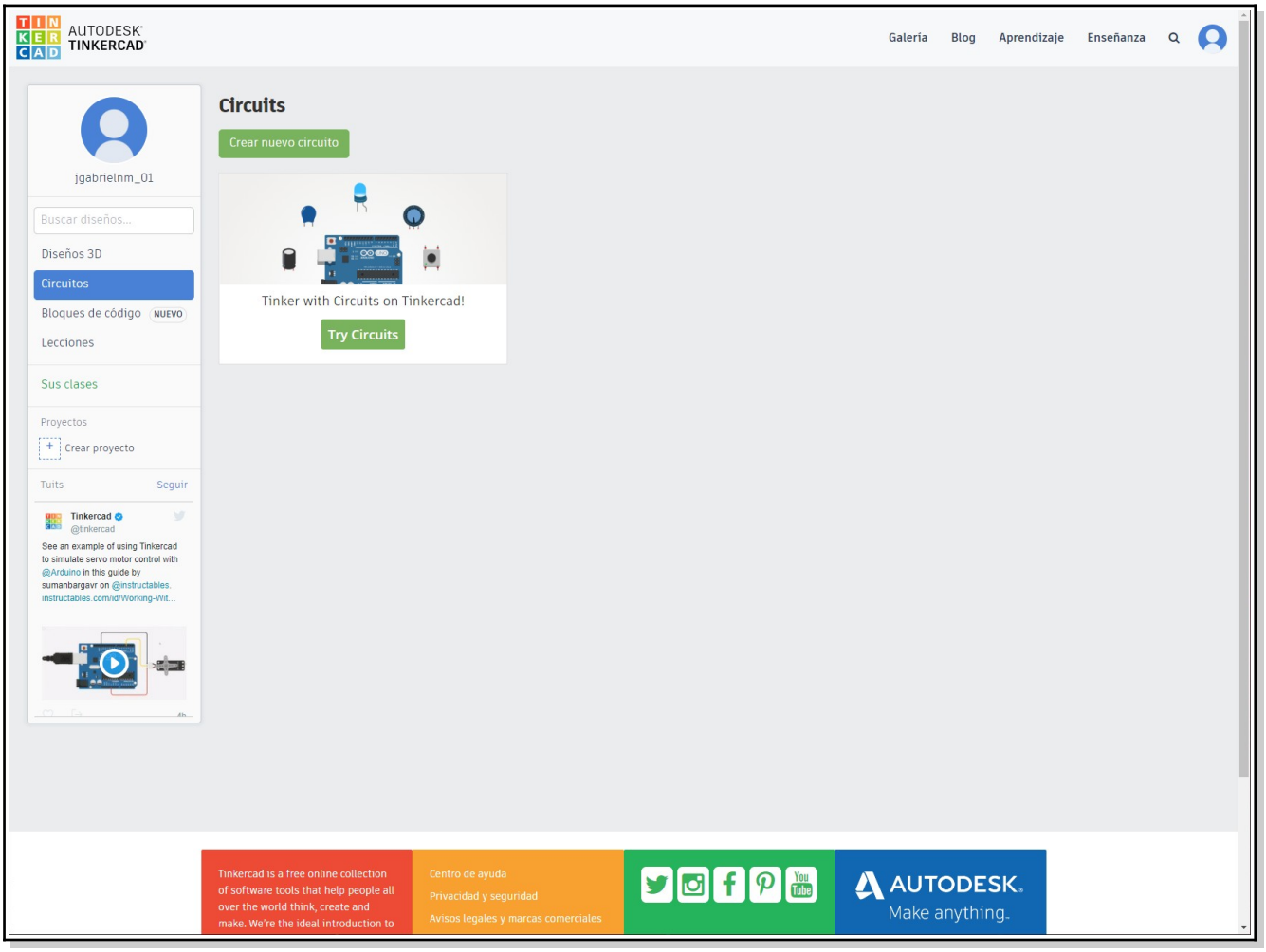



Figura 1. Autodesk Tinkercad. [1]

3. Luego, se selecciona la opción de “Crear un nuevo circuito”. Que desplegará una interfaz similar a la que se observa en la figura 2. Este será el espacio de trabajo en donde se desplegarán los componentes electrónicos a utilizar.

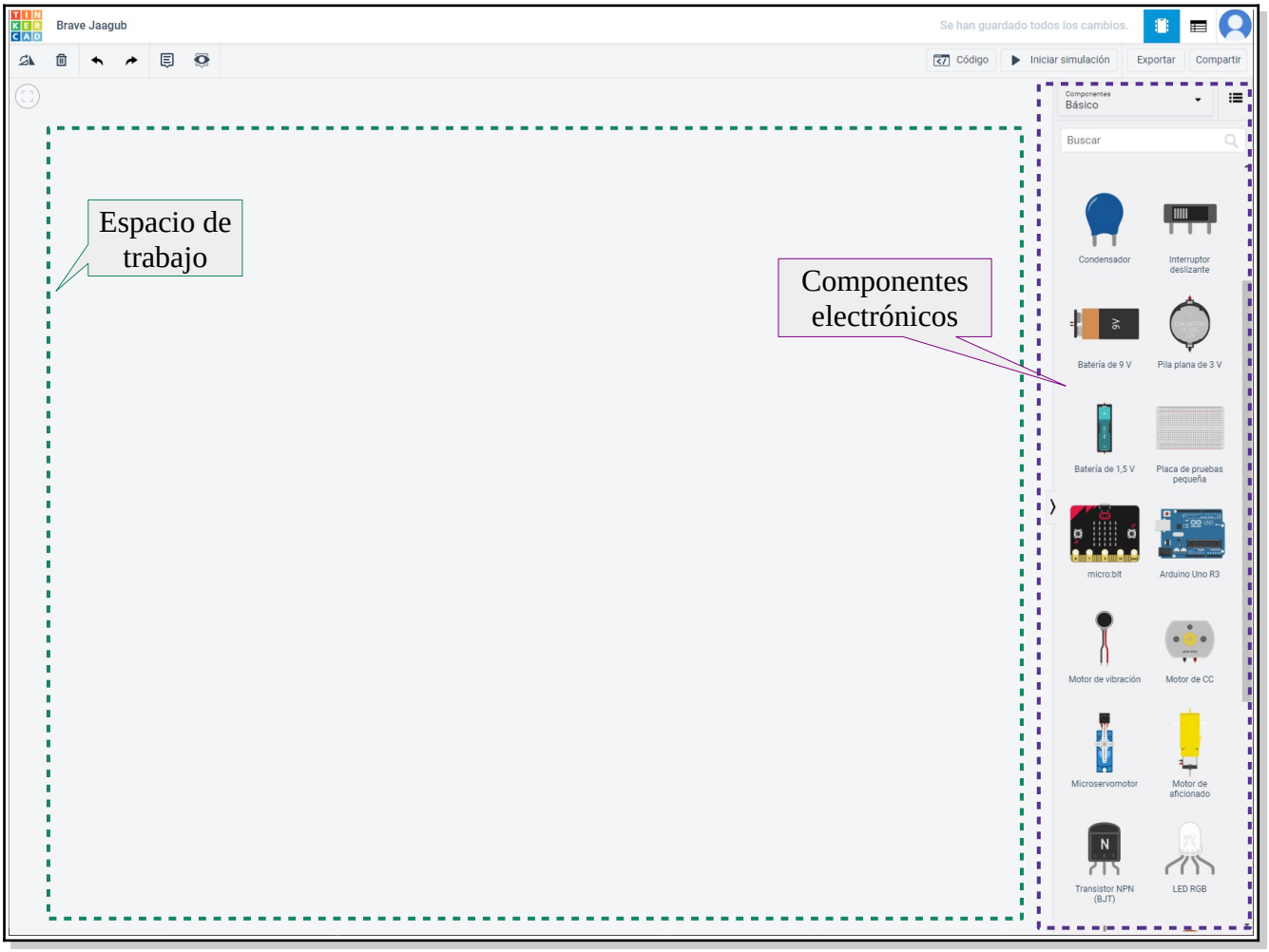



Figura 2. Espacio de trabajo. [1]

4. Ubique los componentes electrónicos que se indican a continuación y según la distribución que se observa en la figura 3.
   a. Un Arduino UNO R3.
   b. Un LED
   c. Una Resistencia de 100 Ω.

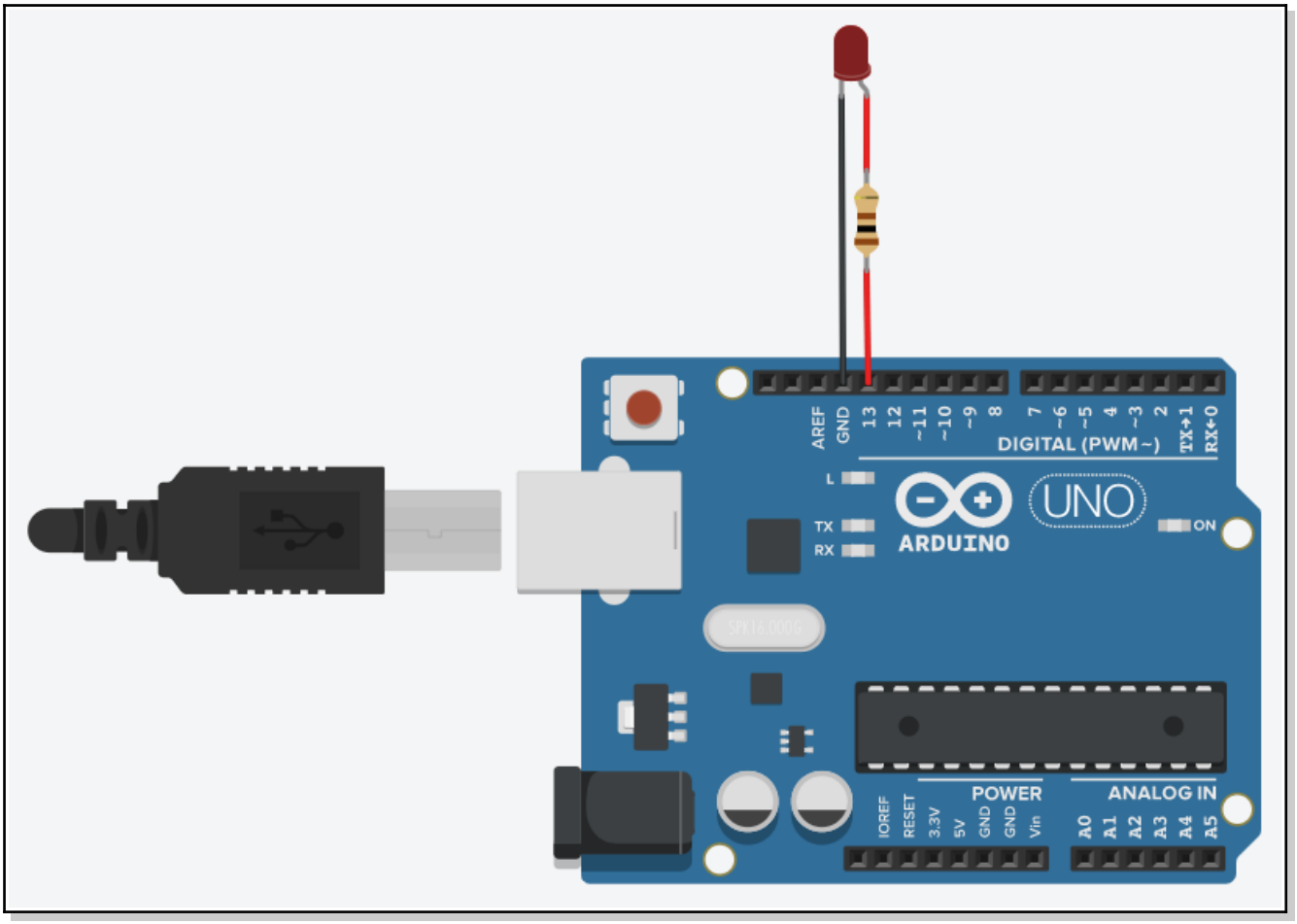



Figura 3. Distribución de los componentes. [1]

5. Cuando se posiciona un componente en el área de trabajo, y se le da un click al componente, se despliega una pequeña ventana con las funciones que se pueden realizar. Para cada tipo de componente se desplegará su correspondiente ventana con opciones. En la figura 4 se muestra la ventana que corresponde a una computadora.

Figura 4. Configuración de una resitencia. [1]

6. Al seleccionar el componente Arduino UNO R3 y luego seleccionar la opción Código, se deberá indicar la opción de Texto, esta acción permitirá ingresarle código C++ al controlador para que realice ciertas funcionalidades. Esta acción se muestra en la siguiente figura.

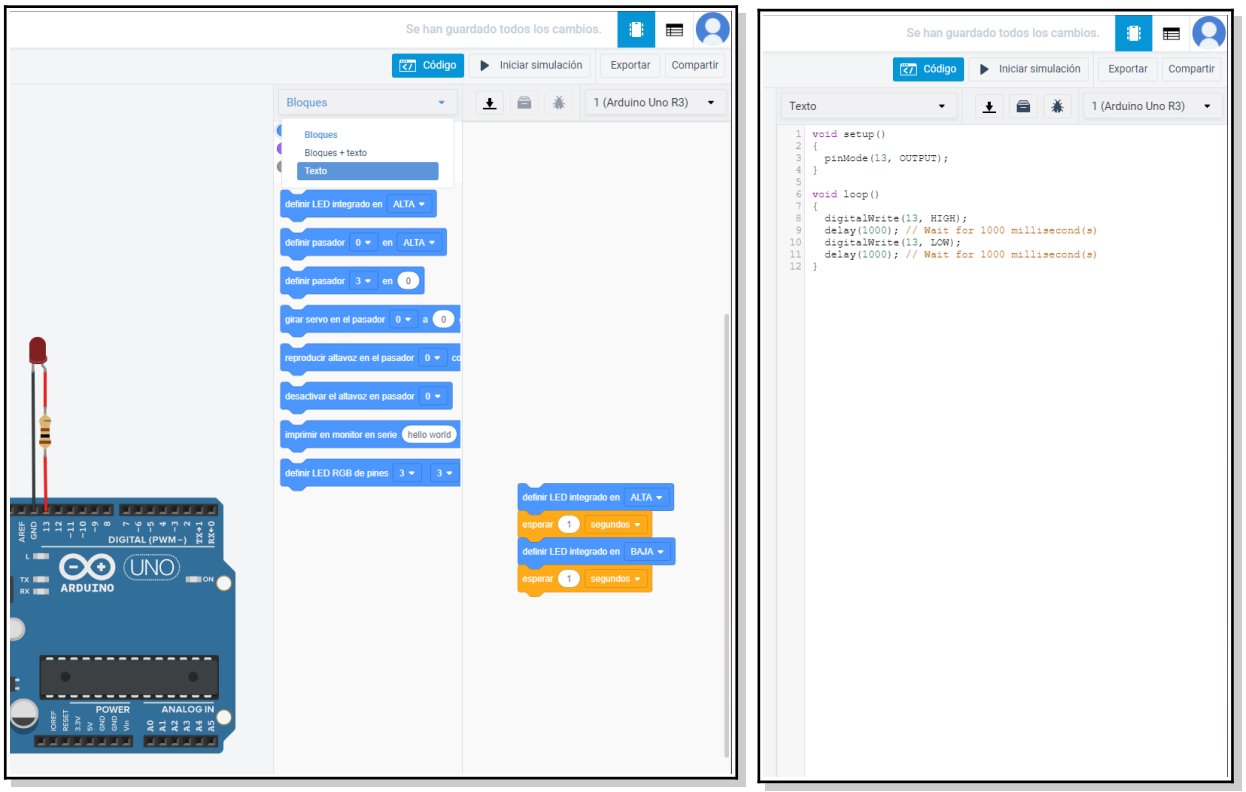



Figura 5. Cambio de programación por bloques a programación en C++. [1]

7. Compruebe que el código que se muestra es el mismo que se indica a continuación y luego seleccione Iniciar simulación.

```
void setup()
{
  pinMode(13, OUTPUT);
}

void loop()
{
  digitalWrite(13, HIGH);
  delay(1000); // Wait for 1000 millisecond(s)
  digitalWrite(13, LOW);
  delay(1000); // Wait for 1000 millisecond(s)
}
```

8. Modifique los valores que se indican en el **delay()** y observe los resultados de estos cambios.

## Segunda Parte – Código Morse

1. En la siguiente tabla se puede observar la representación de cada una de las letras que conforman el alfabeto y su correspondiente representación en Código Morse.

**Tabla 1**

*Letras del alfabeto y su correspondiente representación en Código Morse*

| Letra | Código | Letra | Código | Número | Código |
|---|---|---|---|---|---|
| A | .– | N | –. | 1 | .–––– |
| B | –... | O | ––– | 2 | ..––– |
| C | –.–. | P | .––. | 3 | ...–– |
| D | –.. | Q | ––.– | 4 | ....– |
| E | . | R | .–. | 5 | ..... |
| F | ..–. | S | ... | 6 | –.... |
| G | ––. | T | – | 7 | ––... |
| H | .... | U | ..– | 8 | –––.. |
| I | .. | V | ...– | 9 | ––––. |
| J | .––– | W | .–– | 0 | ––––– |
| K | –.– | X | –..– | | |
| L | .–.. | Y | –.–– | Fin del mensaje | .–.–. |
| M | –– | Z | ––.. | | |

2. Implemente el siguiente código, sustituyendo el código que ya posee el Arduino UNO.
   a. Antes del método **setup()**, ingrese el siguiente código: (puede utilizar la acción de copiar y pegar).

```
String morse[] = {".-" , "-..." , "-.-." , "-.." , "." ,
"..-." , "--." , "...." , ".." , ".---" , "-.-" ,
".-.." , "--" , "-." , "---" , ".--." , "--.-" , ".-." ,
"..." , "-" , "..-" , "...-" , ".--" , "-..-" , "-.--" ,
"--.." , "-----" , ".----" , "..---" , "...--",
"....-" , "....." , "-...." , "--..." , "---.." , "----."
, ".-.-."};

String msg = "Hola Mundo";
int punto = 200; //tiempo de espera de un punto
```

b. En el método **setup()** digite el siguiente código:

```
// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
  Serial.begin(9600);
  msg.toLowerCase();
}
```

Figura 6. Código a digitar en el setup().

c. Luego, en el método **loop()**, digite el siguiente código:

```
// the loop function runs over and over again forever
void loop() {
  uint8_t indice = 0;
  bool saltar;
  for (int i = 0; i < msg.length(); i++) {
    saltar = false;
    char letra = msg[i];
    if (letra >= 97 && letra <= 122) //letras en minusculas
      indice = letra - 97;
    else if (letra >= 48 && letra <= 57) //si son numeros
      indice = letra - 48 + 26;
    else if (letra == 32) {//si es un espacio
      saltar = true;
      Serial.println("Espacio");
      digitalWrite(LED_BUILTIN, LOW);
      delay(5 * punto);
    }
    if (!saltar) {
      Serial.println(morse[indice]);
      codificar(morse[indice]);
    }
  }
  Serial.println(morse[36]);
  codificar(morse[36]);
  Serial.println("-----------------------------------");
}
```

Figura 7. Código a digitar en el loop().

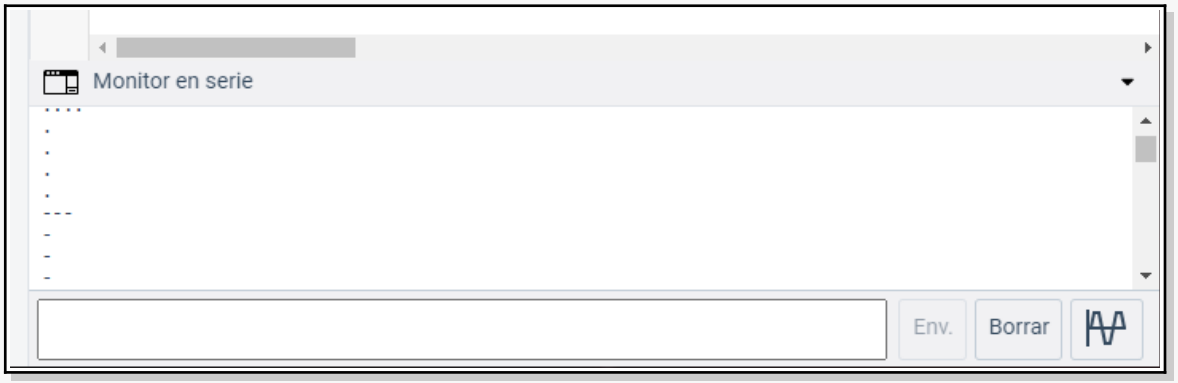
d. Finalmente, cree un método denominado **codificar()** después del método **loop()**.

```
void codificar(String caracter) {
  for (int i = 0; i < caracter.length(); i++) {
    Serial.println(caracter[i]);
    digitalWrite(LED_BUILTIN, HIGH);
    if (caracter[i] == '.') {
      delay(punto);
    } else {
      delay(3 * punto);
    }
    digitalWrite(LED_BUILTIN, LOW);
    delay(punto);
  }
  delay(4 * punto);
}
```

Figura 8. Código a digitar en el método codificar().

3. Una vez ingresado este código, proceda a Iniciar la simulación y observar el resultado.

**Nota:** En la parte inferior de la pantalla podrá desplegar el Monitor en serie, este permite observar lo que imprime el controlador cada vez que se invoca al método **Serial.println(...)**. Por lo que podrá observar el proceso de codificación paso a paso.



**Tercera Parte – Comunicación entre 2 Arduinos**

1. Vuelva a modificar la distribución de los elementos e incorpore un nuevo Arduino UNO R3.
2. Realizar la interconexión entre el dispositivo 1 (Emisor) y el dispositivo 2 (Receptor), tal como se observa en la figura 9 siguiente.

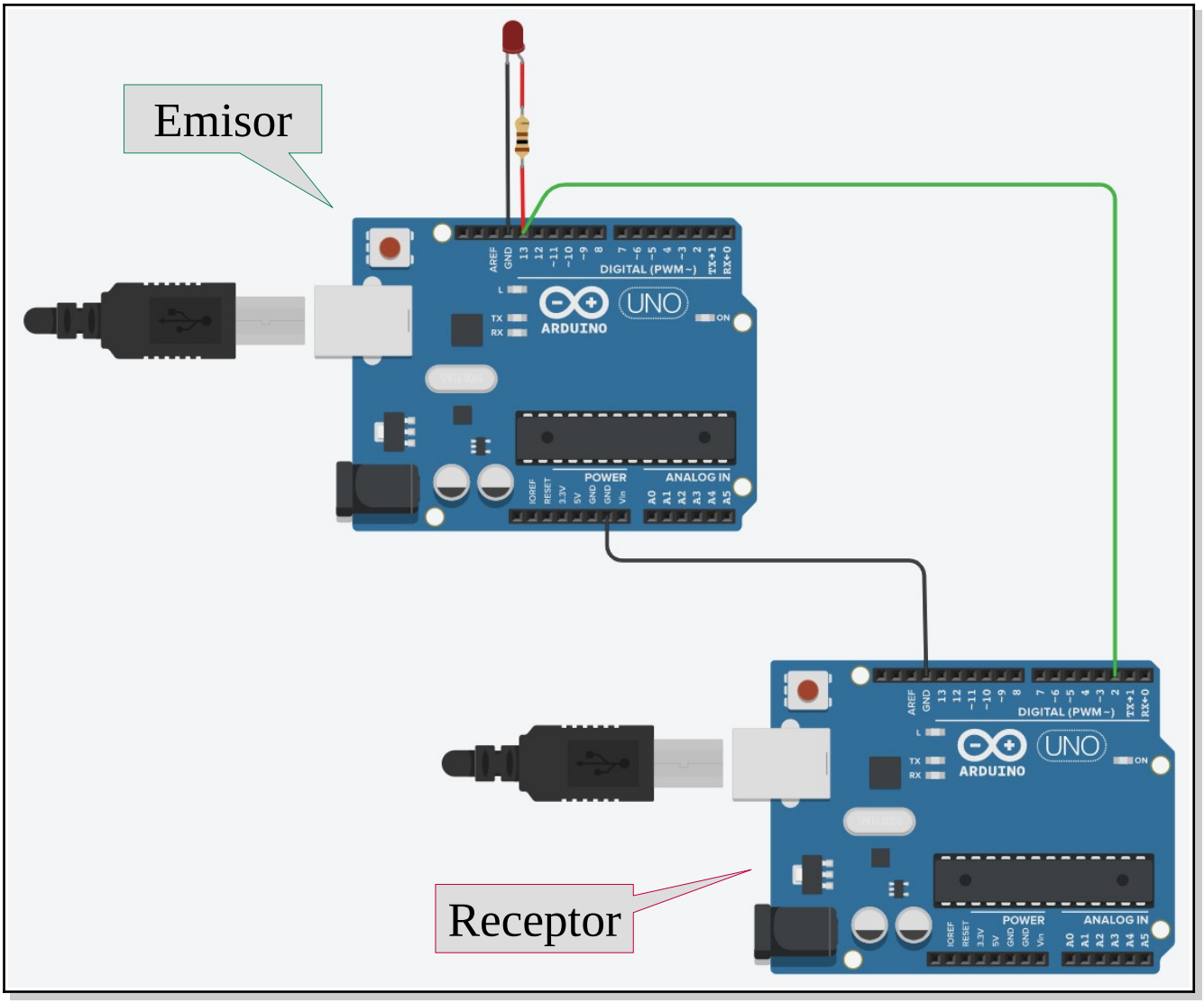



Figura 9. Nueva distribución de los elementos. [1]

3. Seguidamente, vuelva a alterar el código del Arduino, colocado inicialmente, con los siguientes fragmentos de código, este código corresponderá al dispositivo Emisor.

   a. Antes del **setup()** eliminar las variables anteriores o modificarlas según corresponda:

```
String letras[] = {"000000" , "000001" ,
"000010" , "000011" , "000100" , "000101" ,
"000110" , "000111" , "001000" , "001001" ,
"001010" , "001011" , "001100" , "001101" ,
"001110" , "001111" , "010000" , "010001" ,
"010010" , "010011" , "010100" , "010101" ,
"010110" , "010111" , "011000" , "011001" ,
"011010" , "011011" , "011100" , "011101" ,
"011110" , "011111" , "100000" , "100001" ,
"100010" , "100011" , "100100"};

String msg;
uint8_t cero = 100; //tiempo de espera de un
cero
```

   b. Dentro del **setup()**, eliminar la instrucción correspondiente a **msg.toLowerCase();**, porque esta línea pasará al método en el loop().

**Nota:** Observar que en esta nueva parte no se utilizará el Código Morse, debido a que en el Código Morse cada representación del alfabeto posee una diversa cantidad de elementos que conforman la letra. Por ejemplo: la letra E cuenta con un punto y por el contrario la letra Z posee 4 elementos (raya raya punto punto).

Por lo tanto, para esta parte se optó por unificar a cada letra con la misma cantidad de elementos, específicamente 6 dígitos.

c. Dentro del **loop()**, eliminar el contenido y digitar el que se indica a continuación. Esta nueva variación en el código, permitirá que se puede ingresar cualquier mensaje que se quiera transmitir al receptor.

```
// the loop function runs over and over again forever
void loop() {
  if (Serial.available()>0) {
      msg = Serial.readString();
      msg.toLowerCase();

      //Iniciar mensaje
      digitalWrite(LED_BUILTIN, HIGH);
      delay(5 * cero);
      digitalWrite(LED_BUILTIN, LOW);
      delay(cero);

      uint8_t indice = 0;
      for (int i = 0; i < msg.length(); i++) {
        char letra = msg[i];
        if (letra >= 97 && letra <= 122) //letras en minusculas
          indice = letra - 97 + 10;
        else if (letra >= 48 && letra <= 57) //si son numeros
          indice = letra - 48;
        else if (letra == 32) {//si es un espacio
          indice = sizeof(letras)/sizeof(String) - 1;
        }
        codificar(letras[indice]);
      }
      //Finalizar mensaje
      digitalWrite(LED_BUILTIN, HIGH);
      delay(7 * cero);
      digitalWrite(LED_BUILTIN, LOW);
      delay(cero);

      Serial.println("-----------------------------------");
   }
}
```

Figura 10. Código a digitar en el método loop().

d. En el método **codificar()**, ingrese el siguiente código:

```
void codificar(String caracter) {
  for (int i = 0; i < caracter.length(); i++) {
    digitalWrite(LED_BUILTIN, HIGH);
    if (caracter[i] == '0') {
      delay(cero);
    } else {
      delay(3 * cero);
    }
    digitalWrite(LED_BUILTIN, LOW);
    delay(cero);
  }
}
```

Figura 11. Código a digitar en el método codificar().

4. Seguidamente, ingrese nuevamente este nuevo código, pero correspondiente al dispositivo Receptor.

   a. Antes del **setup()** eliminar las variables anteriores o modificarlas según corresponda:

```
String letras[] = {"000000" , "000001" ,
"000010" , "000011" , "000100" , "000101" ,
"000110" , "000111" , "001000" , "001001" ,
"001010" , "001011" , "001100" , "001101" ,
"001110" , "001111" , "010000" , "010001" ,
"010010" , "010011" , "010100" , "010101" ,
"010110" , "010111" , "011000" , "011001" ,
"011010" , "011011" , "011100" , "011101" ,
"011110" , "011111" , "100000" , "100001" ,
"100010" , "100011" , "100100"};

String msg;
uint8_t cero = 1; //tiempo de espera de un cero
uint8_t uno = 3; //tiempo de espera de un uno
uint8_t pinRec = 2;
unsigned long duracion;
```

b. Dentro del **setup()**, eliminar la instrucción correspondiente a **msg.toLowerCase();**, porque esta línea pasará al método en el loop().

```
// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
  pinMode(pinRec, INPUT);
  Serial.begin(9600);
}
```

Figura 12. Código a digitar en el método setup().

c. Dentro del **loop()**, digitar el código que se indica a continuación. A diferencia del emisor.

```
// the loop function runs over and over again forever
void loop() {
  duracion = pulseIn(pinRec, HIGH)/100000;
  //Serial.println(duracion);
  if (duracion == 5 * cero) { //Inicio del mensaje
    Serial.println("Recibiendo mensaje");
    msg = "";
  } else if (duracion == cero)
    msg += "0";
  else if (duracion == uno)
    msg += "1";
  else if (duracion == 7 * cero) { //Fin del mensaje
    Serial.println(msg);
    Serial.println("Mensaje recibido");
    if (msg.length() % 6 == 0) {
      String fragmento = "";
      for (int i = 0; i < msg.length(); i++) {
        fragmento += msg[i];
        if (fragmento.length() == 6) {
          Serial.print(decodificar(fragmento));
          fragmento = "";
        }
      }
    } else {
      Serial.println("El mensaje presenta errores y no se puede decodificar");
    }
    msg = "";
    Serial.println();
  }
}
```

Figura 13. Código a digitar en el método loop().

**Nota:** A diferencia del dispositivo Emisor, el dispositivo Receptor posee en su primera línea en el método loop() un llamado al método pulseIn(), el cuál permite determinar el cambio de voltaje y el tiempo en que ese cambio de voltaje se mantiene. Conociendo estos valores se puede determinar cuando el emisor está enviando un UNO o un CERO

d. De igual manera, así como en el emisor existe un método denominado **codificar()** en el receptor debe existir un método que realice el funcionamiento contrario, por lo que implemente el método **decodificar()** con el siguiente código:

```
char decodificar(String seisBits) {
  int indice;
  char letra;
  bool encontrada = false;
  for (indice = 0; indice < sizeof(letras)/sizeof(String); indice++) {
    if (letras[indice] == seisBits) {
      encontrada = true;
      break;
    }
  }
  if (encontrada) {
    if (indice >= 10 && indice < sizeof(letras)/sizeof(String) - 1) //letras en minusculas
      letra = indice - 10 + 97;
    else if (indice < 10) //si son numeros
      letra = indice + 48;
    else if (indice == sizeof(letras)/sizeof(String) - 1) {//si es un espacio
      letra = 32;
    }
  }
  return letra;
}
```

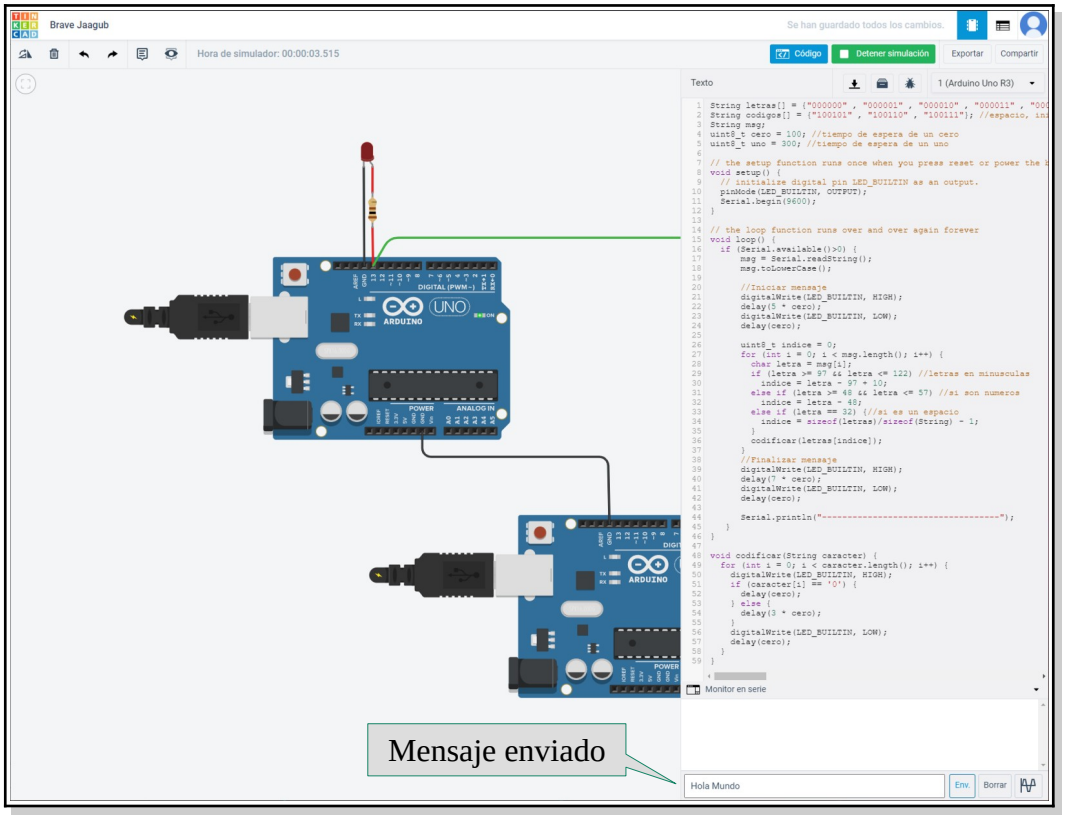
Figura 14. Código a digitar en el método decodificar().

5. Una vez que ya se ha incorporado toda la programación a los dispositivos, tanto Emisor como Receptor, proceda a comprobar el funcionamiento al enviar un mensaje desde el dispositivo Emisor hacia el dispositivo Receptor.

   a. Esta acción se realiza mediante el Iniciar simulación.

   b. Seleccionar el dispositivo Emisor (primer Arduino colocado), seleccionar la opción Código, y luego en Monitor en serie (campo de texto) ingresar el texto que desee.

   c. Una vez que envíe el mensaje desde el Emisor, deberá seleccionar el Receptor (segundo Arduino) y, observar y esperar en el Monitor en serie, que el mensaje se muestre. Esta acción tomará un tiempo, dependiendo del tamaño del mensaje. En la figura 15 se puede observar el resultado.



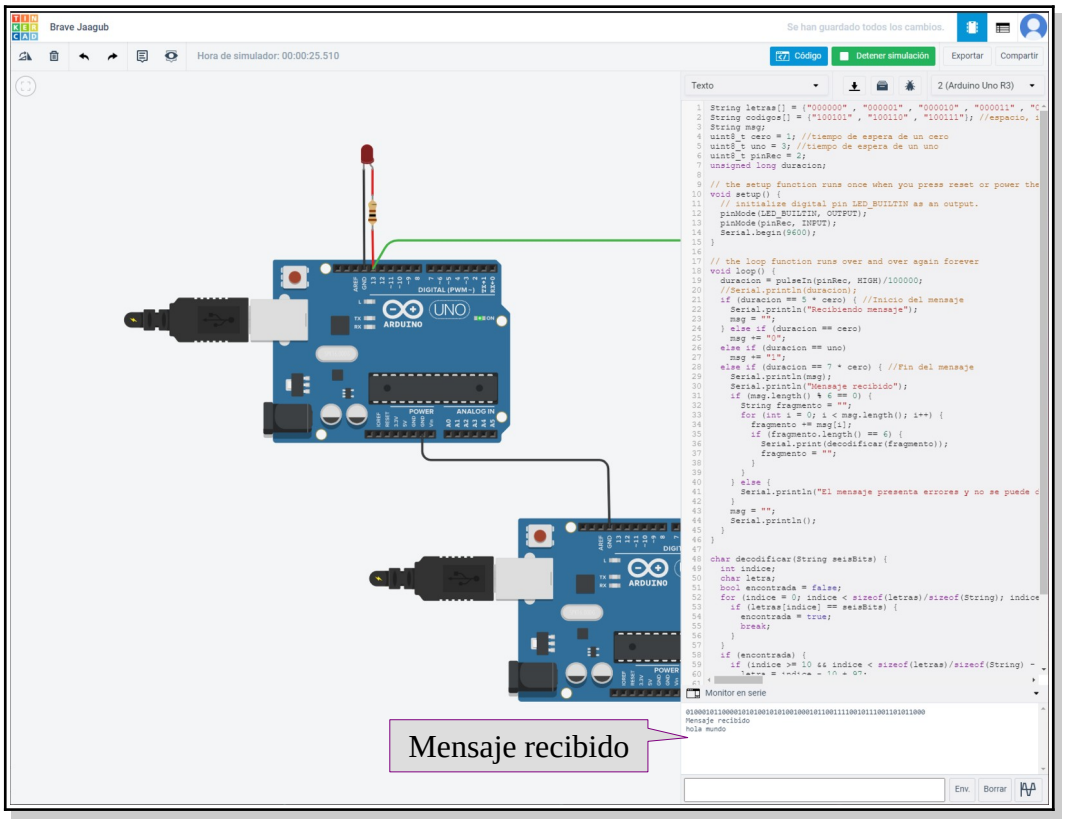




Figura 15. Resultado al enviar y recibir el mensaje. [1]

**Actividad concluida satisfactoriamente.**

**Fin de la Actividad.**

## Referencias bibliográficas

[1] Captura de pantalla del simulador Tinkercad. Disponible en https://www.tinkercad.com/dashboard.